

SPRAWOZDANIE Z PROJEKTU – WARCABY Z MOŻLIWOŚCIĄ GRY Z KOMPUTEREM I LOKALNIE WE DWÓCH.

Patryk Szupryczyński - 331700

Mateusz Wollmann – 331708

1. Wstęp i cel projektu:

Celem niniejszego projektu było zaprojektowanie oraz zakodowanie w pełni funkcjonalnej, konsolowej gry w klasyczne warcaby. Aplikacja miała za zadanie realizować podstawowe zasady gry, a także demonstrować praktyczne zastosowanie zaawansowanych paradygmatów programowania zorientowanego obiektowo (OOP), takich jak:

- **Abstrakcja i enkapsulacja:** Ukrycie wewnętrznej reprezentacji danych i udostępnienie logiki za pomocą publicznych interfejsów (getterzy/setterzy, metody sprawdzające status).
- **Dziedziczenie:** Stworzenie hierarchii klas reprezentujących graczy.
- **Polimorfizm czasu wykonania (metody wirtualne):** Dynamiczne sterowanie przebiegiem tury w zależności od wybranego trybu rozgrywki za pomocą wskaźników klasy bazowej.
- **Dynamiczne zarządzanie pamięcią:** Alokacja i bezpieczne zwalnianie zasobów w trakcie działania programu.

2. Architektura systemu i opis klas:

Projekt opiera się na strukturze plików nagłówkowych (.h) oraz plików źródłowych (.cpp). W procesie kompilacji generowane są również pliki z rozszerzeniem .o (tzw. pliki obiektowe, z ang. *object files*). Są to pliki tymczasowe, które powstają w momencie, gdy kompilator przetłumaczy poszczególne pliki źródłowe (.cpp) na język maszynowy, zrozumiały dla procesora. Nie są to jeszcze gotowe programy, a jedynie "półprodukty" – każdy plik .cpp dostaje swój własny odpowiednik .o (np. main.cpp zamienia się w main.o). Dopiero w ostatnim etapie (tzw. linkowaniu) narzędzie make łączy wszystkie te pliki .o w jeden końcowy plik wykonywalny projekt-warcaby.exe. System ten został podzielony na następujące klasy:

2.1. Klasa bazowa Player

Abstrakcyjna klasa stanowiąca fundament dla każdego uczestnika rozgrywki.

2.2 Klasy pochodne HumanPlayer oraz AIPlayer

Klasy realizujące konkretne implementacje strategii wykonywania ruchów. HumanPlayer odpowiada za interakcję z użytkownikiem. Pobiera współrzędne startowe i docelowe pionka poprzez strumień wejściowy, weryfikując poprawność wprowadzonych danych pod kątem granic planszy i praw własności do pionka, natomiast AIPlayer Reprezentuje algorytm sztucznej inteligencji. Przeszukuje planszę, agreguje wszystkie dostępne dla bota pionki, a następnie automatycznie (i bez udziału człowieka) wybiera oraz wykonuje ruch.

2.3 Klasa Board

Menedżer stanu gry, przechowujący planszę w postaci dwuwymiarowej tablicy wskaźników na obiekty klasy (wymiary 8x8). Inicjalizacja początkowego rozstawienia pionków, czyszczenie pamięci po zbitych elementach, rysowanie siatki w terminalu oraz przemieszczanie obiektów.

2.4 Klasa Piece i struktura Move

Piece reprezentuje pojedynczy pionek na planszy. Przechowuje informacje o swoim kolorze oraz randze (zwykły pionek / damka). Zawiera serce logiczne odpowiedzialne za generowanie wektora legalnych posunięć w oparciu o aktualną sytuację na planszy. Move Lekka struktura (podstawowy kontener danych) przechowująca cztery zmienne całkowite: x1, y1 (współrzędne źródłowe) oraz x2, y2 (współrzędne docelowe).

3. Podział pracy w zespole:

Projekt został podzielony na dwa główne piony: pion logiki gry i struktur danych oraz pion interakcji, sterowania i algorytmiki, żeby równomiernie rozłożyć włożoną pracę.

Mateusz Wollmann wdrożył całą strukturę planszy (Board) oraz logikę i stany pionków (Piece), napisał również funkcję endGame w pliku main.cpp, która sprawdza warunki blokady i kończy grę.

Patryk Szupryczyński stworzył architekturę klas graczy, opierając ją na klasie bazowej Player. Zaimplementował klasę HumanPlayer wraz z błędami wpisywania danych oraz klasę bota AIPlayer, odpowiedzialny był również za strukturę Makefile oraz menu startowe w main.cpp. Praca wspólna objęła stworzenie algorytmu z możliwymi ruchami w klasie Piece oraz integrację i debugowanie całego projektu.

W trakcie prac korzystano z pomocy AI jako asystenta deweloperskiego, pomagało ono w poprawie rozbieżności w plikach .h i .cpp po połączeniu pracy. Narzędzie podsuwało również wiele rozwiązań napotkanych problemów przy tworzeniu projektu. Wspierano się również forami internetowymi oraz filmikami.

Dodatkowo, w celu zwiększenia czytelności i przejrzystości projektu, cały kod źródłowy został opatrzony szczegółowymi komentarzami (w języku polskim). Opisują one przeznaczenie poszczególnych klas, działanie metod oraz kluczowe kroki algorytmów, co znacząco ułatwia analizę struktury programu oraz jego weryfikację.

4. Dokumentacja zrzutów ekranu, prezentacja działania programu:

4.1 Instrukcja uruchomienia programu

Projekt opiera się na programowaniu obiektowym (OOP) i został napisany w standardzie C++11. Do poprawnego uruchomienia i kompilacji projektu wymagane jest jedno z dwóch poniższych środowisk:

A: Środowisko lokalne Win64 (w64devkit): Jest to natywny, przenośny zestaw narzędzi deweloperskich dla systemu Windows (64-bit), który zawiera kompilator GCC (g++), assembler, linker oraz narzędzie automatyzacji GNU Make. Środowisko to działa w pełni lokalnie z poziomu wbudowanej konsoli, nie wymaga instalacji ciężkich środowisk IDE i pozwala na bezpośrednie linkowanie aplikacji z flagami konsolowymi Windows (-WL,-subsystem,console), co eliminuje błędy typu *WinMain*.

B: Internetowy kompilator C++ (Alternatywa): Projekt został zaprojektowany tak, aby w razie potrzeby mógł zostać uruchomiony w chmurze za pomocą wieloplikowych kompilatorów online (np. OnlineGDB, Replit). W takim scenariuszu środowisko internetowe automatycznie emuluje konsolę Linux/Windows, pozwalając na uruchomienie pętli gry bezpośrednio w przeglądarce bez konfiguracji plików.

Wariant A: Uruchomienie lokalne w środowisku Win64 (w64devkit):

Otworzenie konsoli w64devkit, przejście do folderu z projektem przy użyciu komendy cd np. cd C:/Users/Patryk/Desktop/Warcaby

Czyścimy stare pliki komendą “make clean” oraz kompilujemy go za pomocą komendy “make”, a następnie uruchamiamy wygenerowany plik wykonywalny za pomocą “./projekt-warcaby.exe”.

W przesłanej paczce z projektem znajduje się już w pełni skompilowany i gotowy plik wykonywalny projekt-warcaby.exe, który można uruchomić bezpośrednio, bez konieczności ponownej kompilacji kodu źródłowego.

4.2 Menu startowe programu

Po uruchomieniu pliku wykonywalnego użytkownik witany jest tekstowym interfejsem, który oczekuje na wybór trybu rozgrywki. Zapewnia to elastyczność aplikacji bez konieczności rekompilacji kodu.

```
~/Desktop/warcaby $ ./checkers.exe
=====
===   WITAJ W GRZE WARCABY   ===
=====

Wybierz tryb gry:
1. Gra z komputerem (AI)
2. Gra z drugim graczem (Lokalnie)
Twój wybór (1 lub 2):
```

4.3 Widok planszy i mechanizm wyboru ruchu

Poniższy zrzut przedstawia stan graficzny planszy w konsoli. Białe pionki (b) oraz czarne (c) są rozstawione na polach. System wyświetla czytelne indeksy numeryczne (np. 0, 1), wskazując użytkownikowi, na które wolne pola może legalnie przestawić swój zaznaczony pionek.

```
Wybrano gre dla dwóch graczy. Zaczynają Białe!

   0  1  2  3  4  5  6  7
---
0 | b |  | b |  | b |  | b |  |
---
1 |  | b |  | b |  | b |  | b |
---
2 | b |  | b |  | b |  | b |  |
---
3 |  |  |  |  |  |  |  |  |
---
4 |  |  |  |  |  |  |  |  |
---
5 |  | c |  | c |  | c |  | c |
---
6 | c |  | c |  | c |  | c |  |
---
7 |  | c |  | c |  | c |  | c |
---

Twoja tura (Białe).
Podaj współrzędne pionka, którym chcesz się ruszyć.
Podaj X (kolumna 0-7):
```

5. Podsumowanie i wnioski

Projekt zakończył się sukcesem, program poprawnie odwzorowuje mechanikę poruszania się i interakcji pionków warcabach. Zaimplementowano tryb gry przeciwko komputerowi oraz graczowi. Wykorzystanie sztucznej inteligencji otworzyło przed nami nowe możliwości efektywnej nauki i szybkiego radzenia sobie z problemami systemowymi środowiska uruchomieniowego, co stanowi cenne doświadczenie na przyszłość.

Najłatwiejszą rzeczą w projekcie było stworzenie podstawowej struktury planszy 8x8 oraz samo rysowanie tabeli przy użyciu pętli for oraz logika systemu turowego, czyli implementacja naprzemiennego przełączania ruchów pomiędzy dwoma graczami. Natomiast największym problemem była algorytmika ruchów i bić oraz stworzenie możliwości grania z komputerem w tym wielokrotnych bić, integracja kodu z dwóch różnych źródeł.

6. Literatura:

<https://www.youtube.com/watch?v=56v4DKhT9V4>

<https://www.youtube.com/watch?v=uMnh8ZIT88U>

<https://www.youtube.com/watch?v=dZtDZTnO4Ik>

<https://www.cs.yale.edu/homes/jrg94/427/2017f/Assignments/checkers.html>